

Feature: The background runner

From build-plan: feature 8a

Branch: feature/08a-the-background-runner

Status: verified. Feature 8 split into 8a, 8b and 8c on Frank's yes; steps 8a.1 to 8a.3 built, tested and reviewed step by step, each with its re-reviews; no P0 or P1 left open or fixed; decisions 5 and 6 amended with Frank along the way. The checkpoint for the final review.

Goal

Work that has to happen after a booking is saved (its emails, its event in the booked person's Google) stops living in the API's memory. Today it is a promise started after the save: a deploy or a crash in that moment loses it, and a failure is logged once and never tried again, though eight places in the code say "feature 8 retries". After this feature each piece of that work is a job, a row in the same Postgres, added in the same transaction as the booking, cancel or move that needs it. A runner inside the API works the jobs, retries a failure with growing waits, and survives a restart. Nothing changes for Jane or the business when everything works; when Resend or Google fails for a while, the email arrives and the event lands anyway. The texts (8b, 8c) are later jobs on this runner.

In scope

- The runner: started with the API, stopped cleanly on a deploy, its jobs kept in the database. One way to add a job, inside the caller's transaction. One way for tests to run every due job to the end.
- The booking emails as jobs: the confirmation and notification (feature 6), the cancellation pair (7a) and the move pair (7b), one job per email.
- The Google event as jobs: the write when a booking is made (5d), the removal when it is cancelled (7a), the update when it moves (7b).
- The six in-memory trackers removed (F-156), and the carried Google leftovers closed: a move that gives up on the first person's calendar (F-149), two moves inside one follow (F-150), a failed write into the new person's calendar skipping the old event's removal (F-169).

Out of scope

- Texts: the customer's confirmation and reminder (8b), the worker's text (8c).
- Refreshing Google tokens on a schedule, and follow-ups: later features that will use this runner; nothing in 8a needs them.
- A screen showing failed jobs: the owner's leads screens (feature 11) can show them; 8a logs them.

- Any change to what an email or an event says.

Decisions made in the spec

1. **A job is added in the same transaction as the change that needs it.** A booking that rolls back leaves no job; a committed one always has its jobs, even if the API stops the next moment. Nothing is started "after the save" any more. 2. **One job per email and one per calendar change.** A customer email that fails is retried without sending the business's notification again. Each email keeps the idempotency key it has today, so a retry is never a second email. 3. **A job does what is still true when it runs.** It reads the booking afresh, as the senders already do. A confirmation still waiting when the booking was moved or cancelled is not sent (the move or cancel email tells her); this also settles 7b's note that a retried confirmation would carry the new times under its old Resend key. A calendar write for a cancelled booking writes nothing. 4. **Every failure is retried, with growing waits, then given up.** Up to 10 attempts, the waits growing from seconds to about two hours, about three and a half hours in all; never once the appointment has started, when an email or an event change is no use. Giving up logs one line naming the job and the booking, never a customer's details; the job stays in the database, failed. Outcomes that are not failures are not retried: no calendar connected, no email set up, a cancelled booking. 5. **A booking's calendar jobs run one at a time, never in a set order.** Amended with Frank, 2026-10-05, planning 8a.3: they run in one of 16 fixed lanes, the lane named by the last character of the booking's id, so two of one booking's jobs never run at once and two moves close together can no longer race (F-150). A queue per booking was dropped: graphile-worker warns against that many queue names, which would need a cleanup on a schedule, and a failed job's retry waits while a later job of the same lane runs first, so no order holds anyway. Each job does what is still true when it runs (decision 3), which makes the order not matter. Emails need no lane. After the step's review (F-189, Frank, 2026-10-05): 256 lanes, named by the last two characters of the booking's id. A crash mid-job holds its lane for 4 hours, and with it every booking sharing the lane; 256 makes that about 1 booking in 256 instead of 1 in 16, for a few tens of kilobytes of queue rows and no noticeable cost. 6. **A removal carries what it removes.** When a move changes the person, or a booking is cancelled, the job is given the person and the event id at that moment, read inside the transaction (F-149). Writing the new person's event and removing the old one are separate jobs, so one failing never skips the other (F-169). 7. **The runner lives in the API process.** Railway's backend is a persistent Node process (the reason the backend is separate, settled 2026-09-19). `server.ts` starts it before listening and stops it on a deploy, letting running jobs finish; importing `app.ts` never starts it, so tests and the route types stay inert. 8. **Tests run the jobs themselves.** One helper works every due job to the end, replacing the 79 `settled()` waits; a test that wants a failure fakes it as today. 9. **The runner is graphile-worker** (Frank, 2026-10-04, open question 1). A job is added with one SQL call inside our own Drizzle transaction; named queues keep a booking's calendar jobs in order (decision 5); a job key can replace a waiting job, which 8b's reminder will use. Rejected: pg-boss, which needs an adapter to add jobs inside a postgres-js transaction and has no ordered queue per booking; our own table, which would rebuild the locking, waits and crash recovery the library already has. Its one cost: a job left mid-run by a crash (not a deploy, which stops cleanly) is picked up again only after its lock times out.

Build loop

Steps are built one at a time on `feature/08a-the-background-runner`. Each step's plan gets Frank's yes just before it is built. After that yes nothing stops until the review: build, tests, tick the box, the build log entry, commit with the step number and push to the feature branch, `/audit` scoped to the step, then the independent review (`workflow.stepReview: "every"` , `workflow.checkpointCommits: "enabled"`). Findings are talked through after the review; P0/P1 are fixed before the next step. `/complete` makes the merge commit, on Frank's yes.

One package: graphile-worker, installed in `backend` on Frank's yes (2026-10-04, decision 9). It brings its own `pg` driver.

Build steps

- ☒ **8a.1 The runner.**
 - graphile-worker, in `backend` (decision 9, installed on Frank's yes).
 - `backend/lib/jobs/ : enqueueJob(tx, name, payload, options)` that adds a job inside the caller's transaction (options: run after, a queue to run in order, attempts), its payload values ids and numbers only; the runner's start and stop; and a test helper that runs every due job to the end. The job names and their payload types arrive with the jobs that use them, in 8a.2 and 8a.3 (amended after 8a.1's review, F-174).
 - The runner's own tables installed by the API when its runner starts (once a job is defined), and by the test helper, never by hand.
 - `server.ts` starts the runner before listening and stops it on `SIGTERM` and `SIGINT` , letting running jobs finish.
 - Retries as decision 4: the attempt limit, the growing waits, the one log line when a job gives up.
 - **Done when** saved tests, against the local database, with a task only the tests register: a job added in a transaction that rolls back never runs; one that commits runs once; a task that fails twice then succeeds runs three times with growing waits; one that always fails stops at its limit with one log line and stays failed; a job added while no runner is running is worked by the next one started; jobs in one queue run one at a time in order; a runner that stops by itself takes the API down so Railway restarts it, while a stop the API asked for does not (F-173). The backend build passes; the API starts by hand and says the runner is not started while no job is defined (its workers would poll for nothing), so the runner's first start in the API is checked in 8a.2.
- ☒ **8a.2 The emails as jobs.**
 - `bookTime` , `cancelBooking` and `moveBooking` add their email jobs inside their transactions, one per email, in place of the three email trackers, which are removed.
 - Each email job reads the booking afresh and sends one email under the key it has today, recorded on the timeline as today; a confirmation for a booking moved or cancelled meanwhile is not sent (decision 3); nothing after the appointment has started.

- The email tests wait with the helper instead of `settled()` .
- The API started by hand shows the runner working, now that jobs exist.
- **Done when** saved tests: a booking, a cancel and a move each leave their email jobs and send exactly today's emails; a send that fails once is sent on the retry, under the same key, recorded once; the customer's email failing never resends the business's; a confirmation still waiting after a move or a cancel is not sent; no email after the start. Every existing email test passes unchanged in what it checks.
-  **8a.3 The Google event as jobs.** Planned with Frank, 2026-10-05.
 - A booking adds a write, every move adds a move, inside the transactions, in the booking's lane (decision 5), in place of the three event trackers, which are removed (F-156 closed).
 - A write or a move reads the booking when it runs: nothing for a booking gone or no longer confirmed, for a person with no calendar, once the appointment has started, or once a later move has its own job. It saves the event's id only while the booking still has the person and the move number it read, so a change landing during its Google call never gets the wrong calendar's id (F-150).
 - A move to another person, and a cancel, add a take-out carrying the person and the event id at that moment, read inside the transaction (decision 6, F-149). A take-out runs even after the appointment has started, since the event it removes can be at another time, and it is independent of the write, so one failing never skips the other (F-169). A move to the same person stays an update in place when an id is saved. Amended after the step's review (F-187, Frank's yes, 2026-10-05): with no id saved, the move or cancel adds a take-out of the id the write just before it would have used, and the move writes the event afresh under its own id. Narrowed after the re-review (F-190, Frank's yes): any earlier unsaved write was already taken out by its own change, so one id per change is enough and fast moves cost one call each.
 - The Google tests work the jobs with the helper instead of `settled()` .
 - **Done when** saved tests: a move whose first calendar needs reconnecting removes the old event once it is reconnected, within the attempts (F-149); two moves close together, with the middle person's write held mid-call, leave one event, at the last time, in the last person's calendar, with its id saved (F-150); a failed write into the new person's calendar still lets the old event be removed, and the write lands on its retry (F-169); a cancel right after booking, before the event's id is saved, still removes it; two of one booking's jobs never run at the same time; every existing Google test passes unchanged in what it checks. The full backend suite passes several runs in a row, and the API started by hand works a booking's calendar job.

Files / areas

- `backend/lib/jobs/` (new): the job names and payload types, `enqueueJob` , the runner's start and stop, the test helper.
- `backend/server.ts` : start and stop.

- `backend/lib/booking/book-time.ts` , `cancel-booking.ts` , `move-booking.ts` : jobs added inside the transactions.
- `backend/lib/booking/booking-emails.ts` , `booking-event- .ts` : the six trackers, removed.
- `backend/lib/email/send-and-record-emails.ts` , `send-booking-emails.ts` , `send-cancellation-emails.ts` , `send-move-emails.ts` : one email per job.
- `backend/lib/calendar/write-booking-event.ts` , `remove-booking-event.ts` , `move-booking-event.ts` : called by the jobs; the removal takes the person and event id it is given, and the move no longer removes anything.
- The 15 backend test files that wait with `settled()` .

Data / contracts

- **A job:** a name, a payload of ids, when it may run, an optional queue name, attempts made and allowed, its last error (a reason, never the database's own message or a customer's details). Kept in the runner's own tables in the same database; the app's Drizzle schema and migration ledger are untouched.
- **Job names:** `booking_email` ({ `organizationId`, `bookingId`, `kind`, `sequence` } , kind as `BookingEmailKindType`), `booking_event_write` ({ `organizationId`, `bookingId`, `sequence` }), `booking_event_move` ({ `organizationId`, `bookingId`, `sequence` }), `booking_event_remove` ({ `organizationId`, `bookingId`, `personId`, `eventId` } , `eventId` the saved one, or the id the last write would have used when none is saved). The calendar ones run in the lane `booking-event-<last two characters of bookingId>` (decision 5); the names and payloads are settled in 8a.3's plan.
- **No new environment variable** for the API: the runner uses `DATABASE_URL` . `JOBS_SCHEMA` is set only by the tests (8a.2).
- **No route changes** and no change to any answer.

Testing

Backend Vitest against the local database, as 7b: businesses of the test's own, Google and Resend faked, nothing real sent. Tests run jobs with the helper; the runner itself is never started in a test except by the runner's own tests. Each test removes the jobs it made. The frontend is untouched.

Notes for the AI

- The API's queries use `postgres` (postgres-js) through Drizzle; adding a job must use the caller's transaction, whatever driver the runner itself uses.
- Never put a customer's details, a token or a key in a job's payload, its stored error or a log line.
- The idempotency keys today: `booking-confirmation/<id>` , `booking-notification/<id>` , `booking-moved/<id>/<sequence>` , `booking-moved-notification/<id>/<sequence>` and the

cancellation pair as 7a made them. Keep every one.

- The Google ids: `calendarEventIdOf(id, sequence)` ; a write that finds its id already there treats it as written (409), which makes a repeated write safe.
- F-153 (unverified, from 7b): a PATCH of an event deleted by hand may answer 200. Not this feature's work; it stays in the ledger.
- At deploy (Frank, 2026-10-05): set `RAILWAY_DEPLOYMENT_DRAINING_SECONDS=30` on the Railway backend service before the deploy that ships 8a (F-176): Railway's default gives an old API about 0 to 3 seconds after SIGTERM, so a job in flight is cut and then waits about 4 hours for its lock. Check the restart limit at the same time (F-179). Changing Railway is Frank's.
- `npm run db:migrate` builds a fresh database without the runner's tables; the API's start and the test helper install them.
- From building 8a.1, settled planning 8a.3: graphile-worker's documentation warns against queue names with many values (one per booking), so decision 5 uses fixed lanes instead (256 since F-189), and no queue cleanup is needed.
- From building 8a.1, settled in 8a.2: backend test files run in parallel, and working the due jobs takes every due job with a known name. Each Vitest worker now keeps its jobs in a schema of its own (`JOBS_SCHEMA` , set only by `backend/vitest.setup.ts` ; the API uses the library's), cleared before each file, so files still run side by side and never take each other's jobs or the dev API's. Running the files one at a time was tried first: the suite went from 13 to 107 seconds; with a schema per worker it takes 24.
- From building 8a.2: the jobs judge "has the appointment started" by `jobClock` (`backend/lib/jobs/job-clock.ts`); the tests pin it to Friday 2026-10-02, since they book on the fixed Monday after. The setup reads only `DATABASE_URL` from the `.env` , never loading the rest: a test checks the API refusing to start without a setting.
- From building 8a.2: decision 3's rule covers the business's notification as well as Jane's confirmation: once the booking has moved (its sequence above 0) or been cancelled, neither goes; the move or cancel emails tell both. A failed send is now thrown, one line per failed try naming the job and its booking. After 8a.2's review (F-181), a move's emails are not sent once a later move replaced it: the later move's emails say the time that holds.
- After 8a.2's review (F-180): every backend test starts with no job waiting (`vitest.setup.ts` clears the worker's jobs after each test), and the tests' fetch can only reach this machine, whatever a file stubs or unstubs. A job is added with its payload cast through text, so it is stored as an object whichever Postgres driver adds it.
- From building 8a.3: within one run the library looks for its next job before the last job's end (which unlocks its lane) is written, so a job waiting behind another in its lane is left for the next run. The tests' helper runs again while any job of its list is due; in the API the runner takes it on its next poll, about 2 seconds later.
- From building 8a.3: six existing Google tests changed what they check, where the jobs change the facts: the two tests of a move to another person's write and removal are compared without their order. "A booking still confirmed keeps its event" now moves the booking to the same person and sees no

removal, since the removal no longer reads the booking's status. The answer-does-not-wait tests now see Google asked only once the jobs are worked.

- After 8a.3's review (F-187): two more tests changed what they check. "An event never written gets written" now sees a removal and a write instead of an update and a write, and "an event found but never saved gets its id saved" became "an event written but never saved is replaced, never left behind". The move tests' fake Google now keeps each calendar's live events and takes a held write before answering, so tests check the events a calendar holds, not only the calls. After F-188, every test also starts with every lane in its worker's schema unlocked.
- On Windows a stop signal cannot be sent to a process, so the API's stop on SIGTERM is first seen on Railway; the runner's own stop is proved by the tests.

Open questions

1. **Which runner?** Answered by Frank, 2026-10-04: graphile-worker (decision 9).

Implementation walkthrough

What was actually built, by area. The steps above are the plan; the build log records how each step went against it. This is the shape of the code the feature left behind, and the decisions in it that are not visible from the code alone.

backend: the runner (8a.1)

- `graphile-worker` 0.18.0 is the one new dependency (decision 9), in `backend`. It keeps its jobs in its own Postgres schema, outside the Drizzle ledger: `lib/jobs/install-job-tables.ts` installs or updates those tables, the API's runner does the same when it starts, and `db:migrate` never touches them.
- `lib/jobs/enqueue-job.ts`: the one way to add a job, one `add_job` SQL call on the caller's own Drizzle transaction (decision 1), so a change that rolls back leaves no job. The payload is cast through text, so it is stored as an object whichever Postgres driver adds it (F-180). Up to 10 attempts, the library's growing waits (decision 4).
- `lib/jobs/job-runner-options.ts`: what every run shares. A logger that keeps warnings and errors only, first line only, naming the booking a job is for; one line when a job gives up; the plugin that looks for executable task files switched off; and `completeJobBatchDelay` and `failJobBatchDelay` at 0, so a job's end is written before the runner stops (F-185: an exit right after a job finished could leave it locked for four hours, then run it again).
- `lib/jobs/job-task.ts`: wraps every task so a stored or printed failure is always a safe reason, never a database message carrying the query.
- `lib/jobs/start-job-runner.ts`, `server.ts`: the runner starts before the API listens (decision 7), five jobs at a time, and stops with the API on `SIGTERM` or `SIGINT`, letting jobs in hand finish.

Importing `app.ts` never starts it, so tests and the route types stay inert. `exit-when-runner-stops.ts` : a runner that stops by itself takes the API down, so Railway restarts both (F-173); a stop the API asked for does not.

- `lib/jobs/job-schema.ts` , `job-clock.ts` : the schema is the library's own in the API and one per Vitest worker in tests (`JOBS_SCHEMA` , set only by `vitest.setup.ts`); the clock the jobs judge "has the appointment started" by is the real one in the API and pinned to Friday 2026-10-02 in tests.
- `lib/jobs/work-due-jobs.ts` : how tests run jobs (decision 8). It runs the library once, waits until no job is still locked, and runs again while a job of its list is due, because within one run the library can look for its next job before the previous job's end (which unlocks its lane) is written. It throws loudly after 20 runs rather than hide a stuck lane.

backend: the emails as jobs (8a.2)

- `lib/jobs/booking-email-job.ts` , `enqueue-booking-emails.ts` : one job per email (decision 2), added inside `bookTime` , `cancelBooking` and `moveBooking` 's transactions. A job reads the booking when it runs (decision 3): nothing for a booking gone, nothing once the appointment has started, no confirmation or notification once the booking has moved or been cancelled, and no move email once a later move replaced it (F-181). Each email keeps its Resend idempotency key, so a retry is never a second email. A failed send now throws, so the runner tries again.
- The three email trackers in `lib/booking/` are gone.

backend: the Google event as jobs (8a.3)

- Three jobs, each in the booking's lane: `booking_event_write` when a booking is made, `booking_event_move` on every move, `booking_event_remove` on a change of person or a cancel. `enqueue-booking-event-job.ts` is the one way to add them, and always sets the lane.
- `booking-event-lane-of.ts` (decision 5, F-189): 256 lanes, named by the last two characters of the booking's id. Two of one booking's jobs never run at once; no order is promised, since a failed job's retry waits while a later job of its lane runs first. A queue per booking was rejected (the library warns against that many queue names, which would need a scheduled cleanup), and 16 lanes were raised to 256 after the step's review: a crash mid-job holds its lane for four hours, now with about 1 booking in 256 behind it instead of 1 in 16.
- `is-booking-event-job-due.ts` : a write or move does nothing for a booking gone, once the appointment has started (with one log line), or once a later move has added its own job. The removal does not ask: the event it removes can be at another time.
- `lib/calendar/write-booking-event.ts` : saves the event's id only while the booking still has the person and move number it read (F-150), so a change landing during the Google call never gets another calendar's id. `move-booking-event.ts` : the saved event is updated in place; with none saved, the event is written afresh under the move's own id. `remove-booking-event.ts` : takes one named event out of one named person's calendar and forgets it if the booking still names it.

- `move-booking.ts` , `cancel-booking.ts` : a move to another person, a cancel, or a same-person move with no id saved adds a removal carrying the person and one event id read inside the transaction: the saved id, or the id the write just before the change would have used (F-149, F-187, F-190). Any earlier unsaved write was already taken out by its own change, so fast moves cost one call each. The new person's write and the old person's removal are separate jobs, so one failing never skips the other (F-169).
- The three event trackers are gone (F-156): every piece of work after a save is now a job.

Tests

- `vitest.setup.ts` : per worker, its own jobs schema, installed and cleared; after each test no job waits and no lane is locked (F-180, F-188); fetch can only reach this machine; the jobs' clock is pinned. `vitest.config.ts` gives tests 30 seconds (F-183).
- `job-runner.test.ts` proves the runner itself (rollback, commit, retries, giving up, restart, one at a time in a queue, the API exiting with a runner that stops by itself). `booking-email-job.test.ts` proves the emails as jobs. The move tests' fake Google keeps each calendar's live events and takes a held write before answering, so the calendar tests check the events a calendar holds at the end, through crashes, held and failed calls, quick moves and cancels during writes.
- Final count: 599 backend tests and 112 shared, all passing.

Findings

Resolved during this feature and archived with it. IDs carry the feature number; the bare IDs inside each entry are the ones used while the work was live. F-32, F-47, F-58, F-62, F-94, F-95, F-128, F-134, F-137, F-145, F-146, F-153, F-161, F-170, F-171, F-172, F-176, F-179, F-193, F-194 stay in the live ledger.

8a/F-116 [P3] closed - Finding numbers in three code comments added by this feature

File: backend/lib/auth/auth-server.ts:157; backend/lib/auth/login-code-timing.test.ts:2; backend/lib/auth/send-login-code.test.ts:37 **Found:** 2026-10-03 by independent review of feature 6 (scope: 7dc0721..8858d37; lenses: quality, security, performance, tests) **Why it matters:** coding-standards.md (Comments) rules out history in code comments, finding numbers named; F-112 and F-115 were the same slip and were fixed. Three comments written for the F-97 and F-98 repairs still end in "(F-97)" or "(F-98)". After `/complete` archives the ledger these become `6/F-97` , so the bare numbers in the code point at nothing a later reader can find. The comments' reasons are already said in words around them. **Suggested fix:** Drop the three parenthesised numbers and keep the sentences as they are. **Resolution:** Fixed 2026-10-05: the three numbers are gone, and so are two more of the same kind, "(F-92)" in book-time.ts and book-time-resent-while-saving.test.ts; the sentences are unchanged. The "(F-06)" in migrations/0000_adapt_repo_one_tables.sql stays: that migration is already applied. Closed 2026-10-05 by independent review of step 8a.3 (scope: 56f1bae..40f598e): `git grep` for `(F-` and bare

F-NNN across backend, frontend and packages finds no finding number in any code comment outside the applied migration this resolution names; the three originals and the two "(F-92)" are gone.

8a/F-149 [P2] closed - When the follow gives up on the old person's calendar, nothing records that the event is still there, so neither the cancel nor feature 8 can remove it

File: backend/lib/calendar/move-booking-event.ts:46,80 (backend/lib/booking/booking-event-moves.ts:2-3; payload: backend/lib/booking/move-booking.ts:241-246; removal: backend/lib/calendar/remove-booking-event.ts:33) **Found:** 2026-10-04 by independent review of step 7b.3 (scope: 3ba8593..c855db2; lenses: quality, security, performance, tests) **Why it matters:** Who held the event before a move exists only as the in-memory argument to the follow. When the follow fails (F-148's cases, a Google 5xx or timeout, a restart before it runs), or a cancel lands before it reads the row (line 46 returns "nothing", commented "the removal handles it"), the event stays in the old person's calendar while `personId` names the new person, and the cancel's removal asks only the current person's calendar. Probed: Ana's DELETE answering 503 on a move to Mei, then a cancel: the only call is a DELETE in Mei's calendar, `calendarEventId` is cleared, and Ana's 9:00 event is never removed. It keeps Ana busy in Google's free/busy, so her 9:00 is no longer offered to new customers, and nothing can find it: not the booking row, not the `booking_moved` payload (`{ bookingId, fromStartsAt, toStartsAt, sequence }`), not the log line. booking-event-moves.ts says feature 8 tries again; it has nothing to try with. This is the half of F-141 that remains. **Suggested fix:** F-141's first option: save whose calendar holds the event (`calendarPersonId` beside `calendarEventId` , set by the write, cleared by the removal) and have the follow and the removal act on it rather than on the handed-over person and the current `personId` . At the least, add `fromPersonId` and `toPersonId` to the `booking_moved` payload and the spec's contract so feature 8 can redo a person change, and correct the comment at line 46. **Resolution:** Carried to feature 8 (spec, Notes for the AI): its retry job must carry the first person and the event id being removed; a booking column would only half-solve it. Fixed 2026-10-05 in 8a.3: a move to another person and a cancel add a removal job carrying the person and the event id read inside the transaction (the saved id, or the id the event has when none is saved yet), retried on its own. Test: a move whose first calendar needs reconnecting removes the old event once it is reconnected. Closed 2026-10-05 by independent review of step 8a.3 (scope: 56f1bae..40f598e): move-booking.ts:279-284 and cancel-booking.ts:105-113 add a removal job carrying the first person and the event id inside the transaction, and remove-booking-event.ts acts only on those, never on where the booking is now; the test "a move whose first calendar needs reconnecting removes the old event once it is reconnected" passed in 8 of 8 full runs, and the original probe's case (Ana's DELETE failing, then a cancel) now leaves Ana's removal job to retry on its own. A compound race that still leaves an event under an older move's id is F-187.

8a/F-150 [P3] closed - Two moves inside one follow's Google calls leave an orphan event and save the wrong calendar's id

File: backend/lib/calendar/move-booking-event.ts:50,57,82 (backend/lib/calendar/write-booking-event.ts:85-88) **Found:** 2026-10-04 by independent review of step 7b.3 (scope: 3ba8593..c855db2; lenses: quality, security, performance, tests) **Why it matters:** Each follow pairs the person handed over by

its own move with the booking row as it stands when it reads, and `writeBookingEvent` saves its id unconditionally. Probed with Mei's event write held: Jane moved Ana to Mei, then back to Ana, then Mei's answer released. Calls: DELETE plain id at Ana, POST `s1` at Mei, DELETE plain id at Mei (wrong id, Mei holds `s1`), POST `s2` at Ana; the saved id ends as `s1` while the booking is Ana's, and a cancel then deletes `s1` in Ana's calendar, leaving `s1` at Mei and `s2` at Ana for a cancelled booking. A move in the first moment after booking (event created, id not yet saved) can do the same: the PATCH by the plain id misses and `s1` is written beside it. The window is one follow's Google calls (about a second today; the owner's move in features 11 and 12b adds a second actor). Feature 8's retries cannot repair it, since no retry knows which id is right; it belongs to this step's design, but it is rare. **Suggested fix:** Run one booking's follows one at a time (a per-booking chain in booking-event-moves.ts), and with F-149's `calendarPersonId` let each follow compare where the event is with where it should be instead of trusting the handed-over person; save the written id only while `calendarEventId` is still null.

Resolution: Carried to feature 8 with F-149: two moves inside one follow's Google calls; the retry job design covers it. Fixed 2026-10-05 in 8a.3: a booking's calendar jobs run one at a time in its lane, a write or move does nothing once a later move added its own job, and an event id is saved only while the booking still has the person and move number the job read. Test: two moves close together, the middle person's write held mid-call, leave one event at the last time in the last person's calendar. Not closed by independent review of step 8a.3 (scope: 56f1bae..40f598e): the original case holds (Ana to Mei to Ana with Mei's write held leaves one event, `s2` in Ana's calendar, saved), but two changes inside one Google call still leave an event behind when the first change keeps the person: see F-187. Closed 2026-10-05 by independent review of 8a.3's fixes (scope: 40f598e..dea9c59): both forms now hold. The person-change form ("two moves close together leave one event, at the last time, in the last person's calendar") and the same-person form (F-187's four saved cases) passed in 6 of 6 full runs, and the second case in this finding's text (a move in the first moment after booking) is "two moves with the same person during the booking's write leave one event" and "a booking moved before its event was written gets one event". A scratch probe of nine further interleavings ends with one event at the last time in the last person's calendar, its id saved, or none for a cancelled booking (see F-187's closing line).

8a/F-156 [P3] closed - The sixth copy of the "start and settle" background tracker

File: backend/lib/booking/booking-move-emails.ts:9-26 (same body in booking-event-writes.ts, booking-confirmation-emails.ts, booking-event-moves.ts, booking-event-removals.ts, booking-cancellation-emails.ts) **Found:** 2026-10-04 by independent review of step 7b.4 (scope: b56d43a..1524a1b; lenses: quality, security, performance, tests) **Why it matters:** Each module repeats the same `running` set, the `.then/.catch/.finally` chain and `settled()`, differing only in the call and the log line. Every test file that books, moves or cancels must now list up to six `settled()` calls by hand (this step added the new one to four files); a file that forgets one can end the database pool while a send is still writing its `email_sent` entry. Feature 8 will add retries on top of all six. **Suggested fix:** One small helper that takes the work and the log line and returns `{ start, settled }`, with one shared "all background work settled" for tests; keep the six named exports as thin uses of it. **Resolution:** Carried to feature 8 on Frank's call, 2026-10-04: its job runner replaces all six trackers, so a shared helper now would be thrown away. Noted in the spec's Notes for the AI. Stays open until then. Fixed 2026-10-05 in 8a.3: the three

event trackers are removed; with 8a.2's three email trackers, all six are now jobs, and no test waits with `settled()`. Closed 2026-10-05 by independent review of step 8a.3 (scope: 56f1bae..40f598e): no `booking-event-` or `booking-` -emails tracker module is left, `git grep` finds no `new Set<Promise` or tracker import in backend, and the `settled()` calls that remain are each email test file's local name for `workDueJobs()` .

8a/F-169 [P3] closed - A failed write into the new person's calendar skips the removal from the first person's

File: backend/lib/calendar/move-booking-event.ts:82-89 **Found:** 2026-10-04 by independent review of feature 7b (scope: a55c8ee..6c1fa5d; lenses: quality, security, performance, tests) **Why it matters:** On a person change the follow calls `writeBookingEvent` outside any try, and `createEvent` throws on any Google answer but success, as does a token refresh that fails. The first person's removal sits after it, so one failed write leaves the first person's event at the old time even when her calendar is perfectly reachable: she sees an appointment that is no longer hers, and Google's free/busy keeps her busy there, so that time is not offered to new customers. `calendarEventId` was already cleared, so nothing points at the event (F-149's retry gap). F-148 asked for the two calendars to be independent; the repair made the new person's write independent of the first calendar, not the other way round. No test makes the new person's write fail (move-booking-event.test.ts fakes every POST as accepted unless the id was deleted there). **Suggested fix:** Catch the write's failure, run the first person's removal in every case, then rethrow (or log both on one line); add a test where the new person's POST answers 500 and assert a DELETE still reaches the first person's calendar. **Resolution:** Fixed 2026-10-05 in 8a.3: the new person's write and the first person's removal are separate jobs. Test: a failed write into the new person's calendar still lets the old event be removed, and the write lands on its retry. Closed 2026-10-05 by independent review of step 8a.3 (scope: 56f1bae..40f598e): the new person's write is a `booking_event_move` job and the first person's removal a `booking_event_remove` job (move-booking.ts:275-284), so neither depends on the other; the test "a failed write into the new person's calendar still lets the old event be removed" sees the DELETE after a 503 POST and the write land on its retry, 8 of 8 full runs.

8a/F-173 [P1] closed - The API keeps serving after its runner has stopped itself, so jobs pile up unworked until the next deploy

File: backend/server.ts:30 (graphile-worker 0.18.0: dist/runner.js:115-121, dist/main.js:956-963, dist/worker.js:296-301, dist/lib.js:354-362) **Found:** 2026-10-05 by independent review of step 8a.1 (scope: 779512a..17a9118; lenses: quality, security, performance, tests) **Why it matters:** `server.ts` keeps the `runner` only to call `stop()` on a signal; nothing watches `runner.promise` . In graphile-worker 0.18 the runner can stop on its own while the process lives: when a worker cannot mark a job done or failed it "commits seppuku" (worker.js:296-301), the pool then shuts down because "one of the workers exited prematurely" (main.js:956-963), and the runner calls its own `stop()` and resolves `promise` (runner.js:115-121; both branches end in `.catch(noop)` , so it never rejects and nothing crashes). The completion is retried only for codes 40001, 40P01, 57P03, EHOSTUNREACH and ETIMEDOUT (lib.js:354-362); a Postgres restart shows up as 57P01 (admin_shutdown), ECONNREFUSED

or ECONNRESET, which are not retried. Concrete case, from 8a.2 on: Railway restarts Postgres while a confirmation email job is running; its completion fails with ECONNREFUSED; the runner stops and prints one `[jobs] Runner stopping` warning; the API reconnects through postgres-js and keeps taking bookings, each adding its jobs inside the transaction, and none is worked until the next deploy. That is the outage this feature exists to survive ("when Resend or Google fails for a while, the email arrives anyway"), turned silent and indefinite. Reachable only once a task exists (`jobTasks` is empty in 8a.1), which is why it should be settled before 8a.2 puts the emails on it. Found by reading the library source; not reproduced, since that needs the database stopped mid-job. **Suggested fix:** In `server.ts`, after starting the runner: `runner.promise.finally(() => { if (!stopping) { console.error("[jobs] runner stopped on its own; exiting so the API restarts"); process.exit(1); } })`, so Railway restarts the whole process and the new runner picks the jobs up. A test can stop a started runner's pool and assert the hook fires, or the step records the hand check. **Resolution:** Fixed 2026-10-05: `exitWhenRunnerStops` (backend/lib/jobs/exit-when-runner-stops.ts) watches the runner; a stop the API did not ask for logs one line and exits with 1, so Railway restarts the API with a runner. A test stops a real runner from inside and from a SIGTERM: the first exits, the second does not; breaking either branch fails it. While fixing it, a worse case showed: with no job defined yet the library's workers refuse the empty list and exit at once, so 8a.1's API runner died right after "runner working" (F-177). Closed 2026-10-05 by independent review of 8a.1's fixes (scope: 17a9118..882086a): `server.ts:52` attaches `exitWhenRunnerStops(runner, () => stopping)` synchronously after the runner starts, and `stop()` sets `stopping` before it calls `runner.stop()`, so a SIGTERM never reads as a self-stop. In graphile-worker 0.18.0 every stop, the API's or the library's own, goes through the one `stop()` in `dist/runner.js:94-114`: a worker that rejects (seppuku, `worker.js:296-301`) is removed, the pool records the error and shuts down (`main.js:956-963`), `_finPromise` rejects, and the `wp` handler calls `stop()` and ends in `.catch(noop)` (`runner.js:115-117`), so `runner.promise` (`runner.js:121`, `Promise.all([cp, wp])`) resolves and never rejects; `.finally` therefore fires on a self-stop. Run here against `scheduleads_dev`: a runner stopped from outside resolved its promise at once. Railway's restart policy docs: the default is On Failure, which restarts on a non-zero exit, up to 10 times (see F-179 for what that cap means here). The test's first half calls the public `stop()` with a reason rather than making a worker fail, so it proves the watcher and its two branches, and the library source above proves a self-stop reaches it; enough for a two-branch hook. Build, 581 backend tests and `format:check` pass. The aside about F-177 in this resolution does not hold: see F-177.

8a/F-174 [P3] closed - 8a.1's plan lists the job names and their id-only payload types; neither was built and the spec does not say they moved

File: `backend/lib/jobs/enqueue-job.ts:19-23` (spec: `blueprint/context/current-feature.md`, step 8a.1 first bullet and Data / contracts) **Found:** 2026-10-05 by independent review of step 8a.1 (scope: 779512a..17a9118; lenses: quality, security, performance, tests) **Why it matters:** The step's plan says `backend/lib/jobs/` holds "the job names and their payloads (ids only ...)". The step built `enqueueJob(executor, name: string, payload: Record<string, string | number | null>)`: any name and any string value, so a customer's email in a payload compiles and the "ids only" rule rests

on a comment. Leaving the names to 8a.2 and 8a.3, which add the first tasks, is reasonable, but the ticked box says they exist and the spec records no move, which is the kind of silent plan drift the step review is there to catch. **Suggested fix:** Either add the names and payload types from Data / contracts now (a name union and one payload type per name, which `enqueueJob` then takes), or amend step 8a.1 to say they arrive with 8a.2 and 8a.3. **Resolution:** Fixed 2026-10-05 by amending the spec: 8a.1 builds `enqueueJob` with payload values ids and numbers only; the job names and payload types arrive with the jobs that use them, in 8a.2 and 8a.3. Closed 2026-10-05 by independent review of 8a.1's fixes (scope: 17a9118..882086a): step 8a.1's first bullet (current-feature.md:112-117) now describes what was built, `enqueueJob(executor, name: string, payload: Record<string, string | number | null>, options)` in backend/lib/jobs/enqueue-job.ts:19-24, and says where the names and payload types moved; Files / areas and Data / contracts describe the whole feature, so they rightly still list them. 8a.2 gained the API-by-hand check the amendment moved there.

8a/F-175 [P3] closed - The runner's tests leave one job queue row in the database on every run

File: backend/lib/jobs/job-runner.test.ts:54-59 **Found:** 2026-10-05 by independent review of step 8a.1 (scope: 779512a..17a9118; lenses: quality, security, performance, tests) **Why it matters:** The queue test adds jobs in queue `test-<tag>-queue`; `add_job` creates a row in `graphile_worker._private_job_queues` for it. `afterAll` deletes this run's jobs and tasks but not the queue, and the library never removes unused queues by itself. `scheduleads_dev` already holds 8 such rows (`queue_name` like `'test-%'`, counted read-only during this review) while no `test-` task or job is left. Harmless in size today, but it is the same unbounded-queues growth the spec's Notes flag for 8a.3's queue per booking, and the spec's Testing section says each test removes what it made. **Suggested fix:** Add `delete from graphile_worker._private_job_queues where queue_name like 'test-<tag>-%'` to `afterAll`, after the jobs are deleted. **Resolution:** Fixed 2026-10-05: `afterAll` also deletes this run's queue rows; the 8 left by earlier runs were removed from `scheduleads_dev`; a full backend run now leaves 0 jobs and 0 queues. Closed 2026-10-05 by independent review of 8a.1's fixes (scope: 17a9118..882086a): `afterAll` (job-runner.test.ts:60-62) deletes `_private_job_queues` rows matching `test-<tag>-%`, which covers the queue test's `test-<tag>-queue`, after the jobs that reference them are gone. After a full backend run here (59 files, 581 tests), a read-only count of `scheduleads_dev` showed 0 jobs, 0 queues, 0 `test-%` queues and 0 `test-%` tasks.

8a/F-177 [P1] invalid - With no job defined, the API's runner dies right after it starts and nothing says so

File: backend/lib/jobs/start-job-runner.ts:13; backend/server.ts:31 **Found:** 2026-10-05 by the builder while fixing F-173 (scope: step 8a.1) **Why it matters:** `graphile-worker`'s workers assert at least one runnable task; with 8a.1's empty `jobTasks` every worker exited with "No runnable tasks!", the pool shut down and the runner stopped itself. The API had logged "[jobs] runner working" just before, so 8a.1's check by hand proved nothing; with F-173's fix the API would have exited and restarted in a loop. **Suggested fix:** Start the runner only when a job is defined, and say so. **Resolution:** Fixed 2026-10-05:

`startJobRunner` returns null for an empty task list; `server.ts` logs "[jobs] no jobs defined yet: runner not started" and stops or watches the runner only when there is one. A test checks no runner starts for an empty list; removing the guard fails it. Started by hand, the API printed that line and still answered /health 8 seconds later. The runner's first start in the API is checked in 8a.2, when the first jobs exist. Invalid 2026-10-05 by independent review of 8a.1's fixes (scope: 17a9118..882086a): the defect described does not exist in graphile-worker 0.18.0. The "No runnable tasks!" assertion (`dist/taskIdentifiers.js:19`) is thrown inside `getJob` (`sql/getJobs.js:13`), which a continuous worker calls inside a try (`worker.js:90-134`); on an error it logs at debug level ("Failed to acquire job ... contiguous fails") and tries again after `pollInterval` (2 s); only a run-once worker rejects. Run here against `scheduleads_dev` with `taskList: {}`, concurrency 5 and no stop call: after 9 s `runner.promise` was still pending, each of the 5 workers had logged "Failed to acquire job: No runnable tasks!" 5 times at debug, and nothing else. Repeated with the repo's own `jobRunnerOptions({})` and `exitWhenRunnerStops(runner, () => false)`, `process.exit` stubbed: no exit in 9 s. So 8a.1's API runner never died, and F-173's fix would not have looped; what an empty list really does is five workers failing every 2 s, silently, because our logger drops debug lines. The guard in `startJobRunner` and the null runner in `server.ts` are still right for that reason and stay; the wrong reason they carry is F-178.

8a/F-178 [P3] closed - The code and the spec give a library behaviour that does not happen as the reason no runner starts without a job

File: `backend/lib/jobs/start-job-runner.ts:12`; `backend/lib/jobs/job-runner.test.ts:203`;
`blueprint/context/current-feature.md:118-119, 132-134` **Found:** 2026-10-05 by independent review of 8a.1's fixes (scope: 17a9118..882086a; lenses: quality, security, performance, tests) **Why it matters:** The comment in `startJobRunner` ("the library's workers refuse an empty list and exit at once"), the test's comment ("its workers would refuse an empty list and exit") and 8a.1's Done when ("the library refuses an empty list") all state what F-177 found invalid: in 0.18.0 the workers stay up and retry every 2 s, logging only at debug. 8a.2 and 8a.3 build on this runner; a reader trusting the comment would expect an empty or mistyped task list to stop the runner and, through F-173's watcher, restart the API loudly, when in fact it polls silently and works nothing. Separately, the bullet "The runner's own tables installed by the API at start" (`current-feature.md:118`) is no longer true while no job is defined, since the tables are installed only inside `run()`; nothing in 8a.1 adds a job, so nothing breaks before 8a.2. **Suggested fix:** Reword the two comments to the real reason (with no task the workers would only poll and fail every 2 s, silently, at debug level), and amend 8a.1's Done when and its tables bullet to match: no runner and no table install at API start until 8a.2 defines the first jobs. **Resolution:** Fixed 2026-10-05: both comments now say an empty list would leave the workers polling for nothing, silently; the spec's Done when says the same, and its tables line says the API installs them when its runner starts, once a job is defined. Closed 2026-10-05 by independent review of step 8a.2 (scope: f29ce9b..95e47d2): re-read at 95e47d2. `start-job-runner.ts:12` gives the real reason (an empty list would leave the workers polling every two seconds for nothing, silently); `job-runner.test.ts:205` says the same; the spec's tables bullet says the API installs them "when its runner starts (once a job is defined)" and 8a.1's Done when says "its workers would poll for nothing". No old wording ("refuse an empty list") remains in `backend/` or the spec.

8a/F-180 [P1] closed - A failed email job outlives the test that failed it, so a later test works it: the backend suite fails about half the time, and once called the real Resend

File: backend/lib/email/send-move-emails.test.ts:200-207, 326; backend/lib/email/send-booking-emails.test.ts:180-186; backend/lib/email/send-cancellation-emails.test.ts:189-196; backend/vitest.setup.ts:37 **Found:** 2026-10-05 by independent review of step 8a.2 (scope: f29ce9b..95e47d2; lenses: quality, security, performance, tests) **Why it matters:** The "Resend failing" tests in the three email files now leave their two failed jobs in the worker's schema, due again about 2.7 s later (graphile-worker's wait after a first failure, e^1 s). Nothing removes them until the next file starts (`vitest.setup.ts:37`), and every later `settled()` runs `workDueJobs`, which takes every due job in the schema. So the old retry lands inside whichever later test is running then, and its send is counted in that test's `calls`. Seven full runs of `npm run test --workspace=backend` in this review: four failed, each with one test in `send-move-emails.test.ts`: three times "a business without its key moves and sends nothing, and logs it" (line 358: `calls` held another booking's `booking_move` to `jane-...@example.com`) and once "sending a move's emails again sends the very same invite, under the same key" (line 372: the Idempotency-Key of another booking). The plain first run and the builder's passed, so the step's "every step reruns them" gate is a coin flip. The same leftover can also reach the real network: these files unstub `fetch` in `afterEach`, then `afterAll` calls `settled()`. After the first run here, the dev database's `graphile_worker_test_8` schema held `send-booking-emails`' failed jobs (confirmation and notification, attempt 2) whose stored error was "Sending an email failed: validation_error (401)", an answer no stub in the repo gives for a booking email (those tests answer 500 `application_error`; the only 401 stubs are in key tests that make no booking), so the retry went to `api.resend.com` with the fake key `re_primo_send_key`, a test address and the rendered email. Resend refused the key, so nothing was delivered, but the spec's Testing line says "Resend faked, nothing real sent", and on a slow network that `afterAll` waits up to the send's 10 s limit against Vitest's 10 s hook limit. `booking-email-job.test.ts:164-168` already does it right (deletes the schema's jobs after each test). **Suggested fix:** In every test file that fails a send on purpose (at least these three), delete the worker schema's jobs in `afterEach`, as `booking-email-job.test.ts` does, or have each failure test remove its own failed jobs; in `afterAll`, delete leftover jobs instead of working them, or work them before `fetch` is unstubbed. Then run the full backend suite several times in a row to show it is steady. **Resolution:** Fixed 2026-10-05: `backend/vitest.setup.ts` clears the worker's jobs after every test, so no job a test left failing runs inside the next; and the tests' `fetch` throws for any host outside this machine, so an unstubbed `fetch` can never reach Resend or Google again. Evidence: 8 full backend runs, all 586 passing; with the clearing switched off the same failure returned ("a business without its key moves and sends nothing") within 4 runs. The broken job 169 in the dev schema (the builder's own hand check, added with a double-encoded payload) was removed, and `enqueueJob` now casts the payload through text so no driver can store it as a string. Closed 2026-10-05 by independent review of 8a.2's fixes (scope: 95e47d2..f3c983f): the defect is gone. (1) The clearing runs for every test and last: `vitest.setup.ts:54-56` is loaded for every file by `vitest.config.ts`, and Vitest 5's default `sequence.hooks: "stack"` runs it after the file's own and any describe-level `afterEach`. A scratch probe run through the same setup added a job in the test, in a describe `afterEach` and in the file `afterEach`; every next test started with 0 jobs. (2) The fetch guard

holds: the setup assigns it before any file code runs, so `vi.stubGlobal` records the guard as the value to restore and `vi.unstubAllGlobals` puts the guard back, never Node's fetch (the probe saw `globalThis.fetch === guard` after every unstub); string, `URL` and `Request` forms of `api.resend.com` and `googleapis.com`, and a `localhost.evil.com` host, were all refused. No file keeps its own copy of the real fetch. (3) Nothing reaches the API: `tsconfig.json` excludes `vitest..ts`, the build writes no `dist/vitest.setup.js`, nothing imports it. (4) The four email and job files, 4 runs each: with the pre-fix setup (95e47d2, run from a scratch copy) 4 failures in 3 runs, all leftover-job assertions (e.g. "sending a booking's emails again ...": 3 calls, not 2); with the fixed setup 0. (5) After 10 full runs every `graphile_worker_test` schema held 0 jobs (the report found failed jobs left in test_8). (6) The cast: through the API's own Drizzle and postgres-js connection, `json_typeof` gives `object` for both `::json` and `::text::json`, and `enqueueJob` inside a rolled-back transaction stored the payload as an object with `sequence` a number; nothing was left behind. The full suite is still not steady (4 of 10 runs green here), but none of the 6 failures is this defect's shape: they are timeouts and what a timed-out test does to the next one (F-183), and one finished job still listed (F-184).

8a/F-181 [P2] closed - An earlier move's email, retried after a later move, tells Jane and the business a time that no longer holds

File: backend/lib/jobs/booking-email-job.ts:43; backend/lib/email/send-move-emails.ts:34 **Found:** 2026-10-05 by independent review of step 8a.2 (scope: f29ce9b..95e47d2; lenses: quality, security, performance, tests) **Why it matters:** Decision 3 says a job does what is still true when it runs, and the step applies it to the confirmation pair (`sequence > 0` : nothing sent). A move's own job has no such check: `sendMoveEmails` stops only for a cancelled booking, so move 1's emails go even when the booking is already at move 2. Before 8a, "sent late" meant the same moment as the move; now a failed move email is retried for about three and a half hours, so the order can flip. Concretely: Jane moves from 9:00 to 10:00, Resend fails her `booking_move` (job retried after 2.7 s, 7.4 s, 20 s, 55 s, 2.5 min, ...), she moves again to 11:00, move 2's emails go, then move 1's retry succeeds and the last email she gets says "Your booking has moved: 10:00". Her calendar stays right (the invite's SEQUENCE 1 is below 2), but the email text and the business's "moved from 9:00 to 10:00" notice arriving after "moved from 10:00 to 11:00" are wrong. No test covers it; 7b's test "an earlier move's emails, sent after a later move, still carry that move's times" (send-move-emails.test.ts:392) calls `sendMoveEmails` directly and asserts the old behaviour. **Suggested fix:** In the job (or in `sendMoveEmails` when called with `only`), send nothing when the booking's current sequence is above the job's, as the confirmation does (the later move's emails tell both); add a test with a failed move 1 email, a second move, then the retry. Whether 7b's direct-call test keeps its behaviour is Frank's call. **Resolution:** Fixed 2026-10-05: the email job skips a move's emails once the booking's sequence is past that move (a later move replaced it), logging one line; the later move's emails say the time that holds. New test "a move's emails still waiting after a later move are not sent"; removing the skip fails it. The sender called directly still builds any move's emails (7b's F-154 test unchanged). Closed 2026-10-05 by independent review of 8a.2's fixes (scope: 95e47d2..f3c983f): `booking-email-job.ts:38-46` reads the booking's current `sequence` with its start and skips only the two move kinds when it is above the job's own, after the gone and started checks and before any send. A

cancel does not raise `booking.sequence`, so a move's job after a cancel still reaches `sendMoveEmails` 'cancelled check; the confirmation and cancellation kinds are untouched; the log line carries ids only. The new test (`booking-email-job.test.ts:275-293`) makes two moves before working the jobs and expects only the `/2` keys, no job left and the skip line; without the skip the two `/1` keys would be in `calls`, so it guards the repair. It passed in all 10 full runs; `send-move-emails.test.ts:392` (direct call) still passes. The test does not fail a send first, but the job takes the same path on a first try and a retry. Its `jobsOf(id)` read straight after `workDueJobs` is exposed to F-184, if that is real.

8a/F-182 [P3] closed - The spec still says the runner prints the Windows executable-file warning, which 8a.2 turned off

File: `blueprint/context/current-feature.md:249-252`; `backend/lib/jobs/job-runner-options.ts:45` **Found:** 2026-10-05 by independent review of step 8a.2 (scope: `f29ce9b..95e47d2`; lenses: quality, security, performance, tests) **Why it matters:** 8a.2 disables `LoadTaskFromExecutableFilePlugin`, the only source of "Executable file detection not yet supported on win32" (`graphile-worker 0.18.0 dist/plugins/LoadTaskFromExecutableFilePlugin.js:18`), yet the spec's Notes still tell the reader the runner prints it once at start and that it is harmless. Someone checking the API by hand against the spec would look for a line that no longer appears. The note goes into the archive at `/complete`. **Suggested fix:** Reword the note: the plugin is disabled, so nothing is printed; keep the part about SIGTERM on Windows. **Resolution:** Fixed 2026-10-05: the spec note now says only that a stop signal cannot be sent on Windows; the warning line is gone since the plugin is switched off. Closed 2026-10-05 by independent review of 8a.2's fixes (scope: `95e47d2..f3c983f`): the Notes (`current-feature.md`, last bullet) no longer promise the line and keep the SIGTERM part. True for the API: its runner and its table install (`install-job-tables.ts:11`) both go through `jobRunnerOptions`, which disables the plugin. The line still prints in the tests, from `vitest.setup.ts:49` 's own `runMigrations` call, which passes no preset; Vitest 5 shows it only beside a failing file, which is how it surfaced here. Untouched by this range and cosmetic, so no entry of its own.

8a/F-183 [P1] closed - The tests that work jobs run close to Vitest's 5 s limit and time out under ordinary load: the backend suite failed 5 of 10 runs this way

File: `backend/vitest.config.ts:6-8` (no `testTimeout`); `backend/lib/jobs/work-due-jobs.ts:10-12`; `backend/lib/email/send-move-emails.test.ts:127-134`; the same pattern in `send-booking-emails.test.ts`, `send-cancellation-emails.test.ts`, `lib/jobs/booking-email-job.test.ts`, `lib/jobs/job-runner.test.ts` **Found:** 2026-10-05 by independent review of 8a.2's fixes (scope: `95e47d2..f3c983f`; lenses: quality, security, performance, tests) **Why it matters:** Every `workDueJobs` is a whole `graphile-worker runOnce`: the options resolved afresh (a new object each call, so the library's per-options cache at `lib.js:120-126` never hits), a new `pg Pool` opened, and its `end()` not awaited (`lib.js:226-228`). With nothing due, on a quiet machine, one call took 75 to 190 ms. A move test works the due jobs 9 to 12 times (`settled()` alone is three) and renders the emails, so the same test took anywhere from 1.0 to 5.1 s across runs, against Vitest's default 5 s. Ten full runs of `npm run test --workspace=backend` in this review, with Frank's usual dev servers up (Next on 3002, 3100, 3101, `agency-site-app`) and nothing else touching the tests:

runs 2, 3, 4 and 10 green; runs 1, 6, 7, 8 and 9 red with "Test timed out in 5000ms" (1, 7, 8, 1 and 2 timeouts, in send-move-emails, send-cancellation-emails, send-booking-emails, booking-email-job and job-runner); run 5 is F-184. Two files alone (send-move-emails, booking-email-job) still timed out once in two runs ("a booking cancelled since gets no move emails", 5016 ms). Postgres peaked at 59 of 100 connections, so this is time, not a refusal. Not caused by f3c983f: the pre-fix setup gives the same durations on the same files. A timeout also reopens F-180's door: Vitest moves on, but the timed-out test keeps running, so its next `workDueJobs` works the following test's jobs under that test's fetch stub (same schema). Seen in run 7: "a booking, a cancel and a move..." timed out, and the next test, "a send that fails once is sent on the retry...", then counted 3 sends to Jane instead of 2 (booking-email-job.test.ts:229); in job-runner.test.ts "a task that fails twice then succeeds" timed out and the next test found 0 log lines, not 1. The step gate "every step reruns them" is red more often than green. **Suggested fix:** Give the backend tests a longer limit (`testTimeout` in `vitest.config.ts`, say 20 s, or per file for the files that work jobs), and make `workDueJobs` cheaper: one pg Pool per test file passed as `pgPool`, and one options object reused, instead of a new pool and options per call. Then run the full suite 10 times in a row and record the results. **Resolution:** Fixed 2026-10-05: the backend tests get a 30 second time limit for tests and hooks (backend/vitest.config.ts), normal for tests that book, move and work jobs against the real database; with F-184's wait, 10 full backend runs in a row all passed (587 of 587, 32 to 42 seconds each). Not done: sharing one Postgres pool across workDueJobs calls would make each call cheaper, but needs the `pg` driver declared in backend's package.json (today it comes only through graphile-worker), a dependency change that is Frank's call; carried as a note. Closed 2026-10-05 by independent review of 8a.2's second fixes (scope: f3c983f..b4210e9): the defect is gone. `backend/vitest.config.ts:10-11` sets `testTimeout` and `hookTimeout` to 30 s for every backend file (the only Vitest config in `backend`, and the setup file is loaded through it). Ten full runs of `npm run test --workspace=backend` in a row, no dev server started by this review: all ten green, 587 of 587 each, Vitest durations 32.87, 31.63, 32.35, 33.11, 33.48, 32.08, 31.88, 32.50, 33.20 and 32.32 s (33 to 36 s wall). Run 10's JSON timings put the slowest test at 7.2 s ("many simultaneous holds ...", hold-time.test.ts, which would have failed the old 5 s limit) and the slowest job-working test at 3.7 s (send-move-emails.test.ts), so the limit has about four times the worst seen. The longer limit also keeps a slow test from being abandoned while it still works jobs, the way into F-180's shape that this entry described. The hook limit covers F-180's afterAll that waits on a send. The second half of the suggested fix (one pool, one options object) was not done and is not needed to close: the defect was the timeouts. The builder's reason holds: `pgPool` takes a `pg` Pool, and `pg` is not in backend's package.json (only graphile-worker's own dependency), so it is a dependency choice for Frank.

8a/F-184 [P2] closed - workDueJobs may return before its last job is marked done, so a test reading the jobs straight after can see a finished job still waiting

File: backend/lib/jobs/work-due-jobs.ts:11; backend/lib/jobs/booking-email-job.test.ts:213, 288 (graphile-worker 0.18.0: dist/worker.js:262, 283; dist/main.js:376, 879-900; dist/lib.js:226-228) **Found:** 2026-10-05 by independent review of 8a.2's fixes (scope: 95e47d2..f3c983f; lenses: quality, security, performance, tests) **Why it matters:** Run 5 of this review failed once, not by timeout: "a booking, a cancel

and a move each leave their email jobs..." at booking-email-job.test.ts:213 found

`booking_cancellation_notification` still in the table with attempts 1, although its send was already in `calls`. In graphile-worker 0.18 the worker fires `completeJob(job)` and `failJob(...)` without awaiting them (worker.js:262, 283); with the batch delays at their default -1 (main.js:376) nothing tracks those promises, and the pool's `end()` is not awaited either, so `runOnce` can resolve before the last job's delete lands. The other reading is that the job threw after the send (the timeline write); the file mocks `console.warn`, so the reason was lost. Not reproduced: a scratch loop (add a job, `workDueJobs`, read the table) saw it 0 of 40 times on a quiet machine and 0 of 80 beside a full suite run. f3c983f adds one more read of this shape (line 288). **Missing validation:** a reproduction, or the job's `last_error` caught at the moment of the failure. **Suggested fix:** If it recurs: set `completeJobBatchDelay: 0` and `failJobBatchDelay: 0` in the options `workDueJobs` passes, so the pool's shutdown awaits the batchers' release (main.js `terminate`), and prove it with a test. **Resolution:** Confirmed and fixed 2026-10-05: graphile-worker 0.18's worker calls `completeJob` without awaiting it (dist/worker.js), so `runOnce` can return before a job's end is written. `workDueJobs` now waits, up to 5 seconds, until no job in its schema is still locked (a finished job is deleted, a failed one unlocked), and throws if one stays locked. Closed 2026-10-05 by independent review of 8a.2's second fixes (scope: f3c983f..b4210e9): the defect is gone. (1) The claim holds in graphile-worker 0.18.0: the worker fires `completeJob(job)` and `failJob(...)` without `await` (dist/worker.js:262, 283); with the batch delays at their default -1 these are the plain functions at dist/main.js:898-899 and 920-921, whose `release` is null, so `terminate()` (main.js:433-437) has nothing to wait for, and the pool's `end()` is not awaited (lib.js:227). (2) Shown, not only read: a scratch probe whose task row-locked its own job from a second connection for 800 ms made `runOnce` return at 113 to 230 ms with the job still locked, 3 of 3 times; a wait like `workDueJobs` then polled about 50 times and ended 8 to 18 ms after the lock let go, with the job gone. Without the held lock, 60 plain `runOnce` calls never showed the race, which is why the original entry could not reproduce it. (3) The wait is sound: it reads `jobSchema`, the same schema `runOnce` gets from `jobRunnerOptions`, which in the tests is the Vitest worker's own `graphile_worker_test_<pool id>`, so it never waits on another file's or the dev API's jobs; a job's end is one statement (the delete, or the fail that clears `locked_at` and, for a queued job, the queue lock in the same CTE, dist/sql/completeJobs.js, failJobs.js), so "no row locked" cannot be seen before the end commits and the wait cannot end early; jobs whose task is not in the run's list are never locked, so a run with a narrow task list does not wait on them; it is bounded at 5 s and throws, and a rejected end query still surfaces as Vitest's unhandled rejection, so nothing is masked. Every leftover locked row is deleted by the setup's `afterEach`, and `job-runner.test.ts`'s runner tests come after its `workDueJobs` tests. No file that calls it mocks `database.js`. (4) 10 of 10 full runs green (F-183). What this repair does not reach is the same library behaviour in the API's own stop: F-185.

8a/F-185 [P2] closed - A deploy's clean stop can exit before a job that just finished is marked done, so that job stays locked for about four hours

File: backend/server.ts:47-48; backend/lib/jobs/job-runner-options.ts:33-47; backend/lib/jobs/job-runner.test.ts:173-175 (graphile-worker 0.18.0: dist/worker.js:262, 283; dist/main.js:376, 433-437, 898-

899, 920-921; dist/sql/000004.sql:113) **Found:** 2026-10-05 by independent review of 8a.2's second fixes (scope: f3c983f..b4210e9; lenses: quality, security, performance, tests) **Why it matters:** F-184's repair proves that the library ends a job with an un-awaited query. The API's stop has the same gap, and no test or Railway setting covers it: `stop()` runs `await runner?.stop(signal); process.exit(0);`, and `runner.stop()` resolves once the workers have returned, which is right after they fire the end query, not after it lands. A scratch probe did exactly what `server.ts` does (a real runner with the repo's options, one job, the task returns, `stop`, then `process.exit(0)`) and read the table afterwards: the finished job was still there, locked, attempts 1, in 5 of 20 runs. graphile-worker takes such a job again only after its lock is 4 hours old. So a deploy that lands just as a job ends can: run a booking email job a second time 4 hours later (whether Resend drops the repeat depends on its idempotency window for the same key, not checked here); lose a failed try's unlock, so a retry due in seconds waits 4 hours (a confirmation that failed once arrives hours late); and, once 8a.3 puts a booking's calendar jobs in one queue, keep that queue locked too, since the queue unlock is in the same lost statement. Setting `RAILWAY_DEPLOYMENT_DRAINING_SECONDS` (F-176) does not help: nothing kills the process, it exits by itself. Decision 9 and the `server.ts` comment say a deploy stops cleanly; in this window it does not. `job-runner.test.ts:175` reads the jobs straight after `runner.stop()` and is exposed to the same race (it passed in all 10 runs here). **Suggested fix:** Set `completeJobBatchDelay: 0` and `failJobBatchDelay: 0` under `preset.worker` in `jobRunnerOptions`: the ends then go through the library's batcher, whose `release()` waits for every pending end, and `terminate()` awaits it (main.js:433-437, 1094-1104). The same probe with that preset left no job locked in 12 of 12 runs, and with the held-lock probe from F-184 `runOnce` itself returned only after the delete landed. Add a test that stops a runner right after a job's task returns and expects the job gone. With that in place `workDueJobs`' polling wait is no longer needed and can go, or stay as a guard. **Resolution:** Fixed 2026-10-05 as the review proposed: `jobRunnerOptions` sets the library's `completeJobBatchDelay` and `failJobBatchDelay` to 0, so a job's end goes through the batch the runner flushes when it stops, and `server.ts`'s exit after `runner.stop()` no longer leaves a finished job locked (the review's probe: 12 of 12 clean with this setting, 5 of 20 left locked without). The test "a job added while no runner is running is worked by the next one started" checks the job is gone right after `runner.stop()`. Three full backend runs passed afterwards (587 of 587). Closed 2026-10-05 by independent review of step 8a.3 (scope: 56f1bae..40f598e): `job-runner-options.ts` sets `completeJobBatchDelay: 0` and `failJobBatchDelay: 0`, and graphile-worker 0.18.0's `terminate()` awaits both batchers' `release()` (dist/main.js:428-437) before `server.ts:47-48` exits; `job-runner.test.ts:164-176` reads the job gone straight after `runner.stop()` and passed in 8 of 8 full runs.

8a/F-186 [P3] closed - Finding and step numbers in code comments again, two of them added by this range

File: backend/lib/jobs/work-due-jobs.ts:18; backend/vitest.config.ts:2-3; backend/vitest.setup.ts:3; backend/lib/jobs/booking-email-job.ts:3 **Found:** 2026-10-05 by independent review of 8a.2's second fixes (scope: f3c983f..b4210e9; lenses: quality, security, performance, tests) **Why it matters:** coding-standards.md (Comments, lines 358-359) rules out history in code comments, finding numbers and step

numbers named; F-116 is the same slip, still open. This range adds "(F-184)" (work-due-jobs.ts:18) and "(F-183)" (vitest.config.ts:3). The previous range added "(F-180)" (vitest.setup.ts:3) and "(F-181)" (booking-email-job.ts:3), which its review did not raise, and vitest.config.ts:2 carries "(8a.2)". After `/complete` the ledger's numbers become `8a/F-...`, so the bare ones point at nothing; each sentence already says its reason in words. **Suggested fix:** Drop the five parenthesised numbers and keep the sentences. **Resolution:** Fixed 2026-10-05: the finding and step numbers are gone from the comments in work-due-jobs.ts, vitest.config.ts, vitest.setup.ts, booking-email-job.ts and its test, send-and-record-emails.ts and the three booking files; booking-email-job.ts's header is rewrapped to the usual width. Closed 2026-10-05 by independent review of step 8a.3 (scope: 56f1bae..40f598e): the five numbers are gone (`git grep` finds no `(F-` or `8a.` step number in backend, frontend or packages comments), and the comments this range adds carry none either.

8a/F-187 [P2] closed - Two changes inside one Google call still leave an event behind when the first one keeps the person: an extra event, or one for a cancelled booking

File: backend/lib/booking/move-booking.ts:235,277; backend/lib/booking/cancel-booking.ts:111; backend/lib/calendar/move-booking-event.ts:50 (save that loses the race: backend/lib/calendar/write-booking-event.ts:87-100, move-booking-event.ts:61-71) **Found:** 2026-10-05 by independent review of step 8a.3 (scope: 56f1bae..40f598e; lenses: quality, security, performance, tests) **Why it matters:** When no id is saved, a move or cancel names the event as `calendarEventIdOf(bookingId, row.sequence)`, the id the current move number would give. That is only right if the event was written under that number. A same-person move whose job has not saved its id yet leaves the event under the previous number's id, known only to that move's job payload; the next change names the wrong id, and the older event is never touched again. Probed with a scratch test (this file's Google fake, extended to keep each calendar's live events and to take a POST before its answer is held), 1 of 1 each: (A) the booking's write held mid-call, two moves with Ana to 10:00 and 11:00: Ana's calendar ends with the plain id at 9:00 and `s2` at 11:00, two events for one booking; (B1) write held, move with Ana to 10:00, then a cancel while that move's PATCH of the plain id is mid-call: the removal deletes `s1`, the plain id stays at 10:00 for a cancelled booking; (B2) the same with the move's PATCH failing once (503) and the cancel landing before its retry: the plain id stays at 9:00, cancelled; (C) as B1 but a move to Mei instead of the cancel: Mei gets `s2`, Ana keeps the plain id at 10:00 while the booking is Mei's. B2 needs only one transient Google error after a move in the first second after booking; the others need two changes inside one call (the owner's screens, features 11 and 12b, add a second actor). A cancelled appointment left in a worker's calendar can send them to a customer's house, and it holds that time busy in free/busy. This is F-150's title in its same-person form; the step's test covers only the person-change form, and its fake answers a held POST before recording it, so it cannot see an event Google took but answered late. **Suggested fix:** Make the id the event may have survive same-person moves: for example save the intended id on the booking inside the transaction that adds the write or move job (the write already knows it before calling Google), so the next change always names the event that exists; or have the removal and the same-person update try every unsaved id since the last person change (404 and 410 already count as gone). Add a test with a fake that records a POST or PATCH before holding its answer, asserting the live events per calendar. **Resolution:**

Fixed 2026-10-05 on Frank's yes: a move that keeps the person updates the saved event in place only when an id is saved. With none saved, as while an earlier write has not saved its id, the move or cancel adds a removal carrying the booking's move number, which takes out every id the event may have had (one never written is gone already, as Google answers), and the move's job writes the event afresh under its own id. The move job no longer takes an event id. Tests, with a fake Google that keeps each calendar's live events and takes a held write before answering: the reviewer's four cases (two same-person moves during the booking's write; a cancel during a same-person move's write; a cancel while that write waits for its retry; a move to another person during it) each end with one event at the last time, or none for a cancelled booking. Three of the four fail on the code before the fix; the retry case passes there too, since the old code moved in place instead of writing. Closed 2026-10-05 by independent review of 8a.3's fixes (scope: 40f598e..dea9c59): an event is now written only under the move number the write reads (write-booking-event.ts:65-69), its id goes unsaved only when a change lands during the call (87-99), and that change reads no saved id, so it adds a removal for that person and number (move-booking.ts:278-290, cancel-booking.ts:104-113); a write never uses a number at or below one a removal named in that calendar, so no path writes an id this booking deleted there (the saved tests' fake refuses every deleted id and they pass). The four saved cases pass in 6 of 6 full runs; on the code before the fix, run in a scratch copy, three fail (two of them only by the 30 s limit, waiting for a write the old code never makes). A scratch probe with a Google that keeps live events and takes each call before its answer ended correctly in nine more interleavings, six of which fail on the code before the fix: 20 moves faster than the jobs, alternating people and with one person; lost answers at every write across Ana, Mei, Ana; a person change and a same-person move during held writes, then a cancel; moves while a removal is mid-call; an event deleted by hand with a move during the rewrite; a cancel during a PATCH; a crashed write whose lock ran out with a move waiting; a cancel while the move waits for its retry after a call Google took. The cost of the removal's range is F-190, the retry test's gap F-191.

8a/F-188 [P3] closed - A test that ends with a calendar job still running leaves its lane locked in the worker's schema, and later runs fail for four hours

File: backend/vitest.setup.ts:55-59; backend/lib/jobs/work-due-jobs.ts:34-46 (graphile-worker 0.18.0: dist/sql/completeJobs.js, failJobs.js, resetLockedAt.js) **Found:** 2026-10-05 by independent review of step 8a.3 (scope: 56f1bae..40f598e; lenses: quality, security, performance, tests) **Why it matters:** The setup clears `_private_jobs` after each test but not `_private_job_queues`. A queue is unlocked only in the same statement that ends its job (`... from j where job_queues.id = j.job_queue_id`), so a job deleted while it runs can never unlock its lane; only `resetLockedAt` frees it, after 4 hours. The lane names are now fixed (`booking-event-0` to `-f`) and the per-worker schemas persist between runs, so a test that times out or is stopped with a held Google answer (F-95's shape) poisons that lane for every later run on that Vitest pool id. Probed: a booking's write held mid-call, the jobs deleted as `afterEach` does, the hold released: the lane row stayed locked, and a later job in the same lane made `workDueJobs` throw "Jobs were still due after 20 runs" (the probe then unlocked it by hand). Roughly one booking in 16 per test would then fail on that worker, a flake that looks random for hours. **Suggested fix:** In `vitest.setup.ts`, clear the lanes with the jobs (`update ... _private_job_queues set locked_at`

`= null, locked_by = null` , or delete the queue rows), before the file and after each test. **Resolution:** Fixed 2026-10-05 as suggested: vitest.setup.ts unlocks every lane in the worker's schema along with clearing its jobs, before each file and after each test. Closed 2026-10-05 by independent review of 8a.3's fixes (scope: 40f598e..dea9c59): vitest.setup.ts:57-62 clears `locked_at` and `locked_by` on `_private_job_queues` , whose `is_available` is generated from `locked_at is null` (graphile-worker 0.18.0 dist/generated/sql.js:1164), the column getJobs reads. Probed in a scratch copy: a test ending with a booking's write held mid-call left 1 locked lane, and the next test started with 0; with the setup from 40f598e the next test started with 1. After every run and probe of this review, all 16 job schemas held no job and no locked lane.

8a/F-189 [P3] closed - A crash mid-job now holds a sixteenth of every business's calendar jobs for four hours, not only that booking's

File: backend/lib/jobs/booking-event-lane-of.ts:4-6; blueprint/context/current-feature.md (decisions 5 and 9) (graphile-worker 0.18.0: dist/sql/getJobs.js queue clause, dist/sql/resetLockedAt.js) **Found:** 2026-10-05 by independent review of step 8a.3 (scope: 56f1bae..40f598e; lenses: quality, security, performance, tests) **Why it matters:** Decision 9 accepts that a job left mid-run by a crash waits for its 4-hour lock. With 16 lanes shared by every tenant, its lane stays locked too: `getJobs` takes a queued job only while its queue row is available, and `resetLockedAt` frees the row only after 4 hours. So every calendar job of every business whose booking id ends in that character waits 4 hours, and a write or move for an appointment starting within that time is then skipped as started (decision 4): the worker never gets those events. F-188's probe shows the same mechanism in the tests: one locked lane holds another booking's job. Decision 5's amendment weighed queue-name count and retry order, not this blast radius. Rare (a crash, not a deploy, while one of up to 5 jobs is in a Google call), so a note rather than a defect. **Suggested fix:** Record the cost in decision 5 as accepted, or narrow it: more lanes (the last two characters give 256) shrink the share, and the lane lock could be released at start-up for workers known to be gone. **Resolution:** Fixed 2026-10-05 on Frank's call: 256 lanes, named by the last two characters of the booking's id (bookingEventLaneOf). A crash mid-job still holds its own job and lane for 4 hours (decision 9's accepted cost), but now about 1 booking in 256 shares that lane instead of 1 in 16. The cost is up to 256 small queue rows, made as each lane is first used. Closed 2026-10-05 by independent review of 8a.3's second fixes (scope: dea9c59..7f10bdc): `bookingEventLaneOf` takes `slice(-2)` (booking-event-lane-of.ts:6), and the only place a booking id is made is `randomUUID()` (book-time.ts:274), whose last two characters are lowercase hex, so the lanes are exactly `booking-event-00` to `-ff` ; the comment says the same. Nothing else counts lanes: every calendar job takes its lane from this one function (enqueue-booking-event-job.ts:21), and the tests' setup unlocks every queue row whatever its name (vitest.setup.ts:57-62). The spec's decision 5, job contract and Notes say 256. The full backend suite passed 6 of 6 runs, and every job schema held no job and no locked lane afterwards.

8a/F-190 [P3] closed - With no id saved, a removal asks Google once per move number, so moves made faster than the jobs cost calls that grow with the square of the moves

File: backend/lib/jobs/booking-event-removal-job.ts:25-27; backend/lib/calendar/remove-booking-event.ts:25-27 (added by: backend/lib/booking/move-booking.ts:270-290, backend/lib/booking/cancel-booking.ts:102-113) **Found:** 2026-10-05 by independent review of 8a.3's fixes (scope: 40f598e..dea9c59; lenses: quality, security, performance, tests) **Why it matters:** With `eventId` null the removal deletes `calendarEventIdOf(bookingId, k)` for every `k` from 0 to `sequence`, one call after another in the booking's lane, and a failed call starts the list again on the retry. Every move made before the last move's job has saved its id adds such a removal (a move to another person always clears the id), so `n` moves that outrun the jobs cost $n(n+1)/2$ calls. Probed in a scratch copy of the backend with a fake Google that keeps live events: 20 moves alternating Ana and Mei, the jobs worked only after the last, made 210 DELETE calls; 20 same-person moves while the booking's write was held made 210 too (the code before the fix: 20 and 0). The move route is public with no rate limit yet (as F-62 notes for the times route), and the lane is one of 16 shared by every business (F-189), so a scripted customer holds a sixteenth of all calendar jobs while Google answers, and a per-user quota error restarts the list. The range buys nothing the change's own id does not: an event is written only under the move number and person `writeBookingEvent` reads, and only while no id is saved (write-booking-event.ts:20-25, 65-69); its id goes unsaved only when a change lands during the call (87-99); that change reads no saved id, since the lane lets no other job of the booking save meanwhile, so it adds a removal for that person and `row.sequence`, which names exactly that id. With the job changed to delete only `calendarEventIdOf(bookingId, sequence)`, the 27 move tests (the four F-187 cases among them), every interleaving probed under F-187 and the full backend suite (609 tests with the probes) all pass, with 20 DELETE calls in both cost probes. Only a second runner working the same booking at once (a lock that ran out on a live process) would need the earlier ids. The comments at move-booking.ts:271-273 ("under any earlier id"), cancel-booking.ts:102 ("every id it may have") and booking-event-removal-job.ts:2-4 state the wider claim. **Suggested fix:** When no id is saved, delete only the id of the move number the job carries, and say so in those three comments. Or, to keep the range as cover against a doubled run, bound it (for example to the ids since the last person change). **Resolution:** Fixed 2026-10-05 on Frank's yes, as suggested: a removal carries one id again, the saved one or, with none saved, the id the write just before the change would have used (`calendarEventIdOf(bookingId, row.sequence)`, read in the transaction); any earlier unsaved write was taken out by its own change. Test: five quick moves between Ana and Mei during a held write make five DELETE calls and leave one event at the last time; on the range version they made 15. Closed 2026-10-05 by independent review of 8a.3's second fixes (scope: dea9c59..7f10bdc): the one id holds. An event is written only under `calendarEventIdOf(bookingId, row.sequence)` for the person read in the same query (write-booking-event.ts:20-25, 65-67), so an id always belongs to one move number and that move's person; it is saved only while that number and person still hold (87-100), and every change reads the row under `for update` (move-booking.ts:203, cancel-booking.ts:50), so a change after an unsaved write sees no id and names exactly that person and number (move-booking.ts:280-290, cancel-booking.ts:106-113). Every write sits in the booking's lane, and the lane lock lives in the database (graphile-worker 0.18.0 dist/sql/getJobs.js:95-129), so two runners never write one booking at once; a crashed job's lock is reset with `run_at = greatest(run_at, now())` (dist/sql/resetLockedAt.js:8), so the jobs queued behind it run first, and its retry is skipped once a later move exists (is-booking-event-job-due.ts:27). Writes only

use the current number, which no removal has named yet, so no path writes an id this booking deleted in that calendar. Probed in a scratch copy with the move tests' fake extended, 11 of 11 pass on this range: a write crashed after Google took the plain id, with the 4-hour lock expired by the library's own SQL, then a same-person move; then Ana, Mei and a cancel; the crashed write retried before the jobs behind it; no change at all (its retry finds its own event by 409); a crashed move job with `s1` taken in Mei, then back to Ana; a write taken then answered 503 with a move before its retry; every write taken then lost across Ana, Mei, Ana and a cancel; a person change, and a same-person move with nothing saved, each while a removal is mid-call; an event deleted by hand, its rewrite taken then lost, a cancel before the retry; 20 alternating moves during a held write (20 DELETE calls, one event). Each probe also asserts no POST of an id after a DELETE of it in that calendar. Five of the eleven fail on 40f598e. The quick-moves test fails on dea9c59 with 15 DELETE calls instead of 5 (rerun here), and the full suite passed 6 of 6 runs (599 tests).

8a/F-191 [P3] closed - The retry test's failing write is refused before Google takes it, so it cannot fail for the case it is named for

File: backend/lib/calendar/move-booking-event.test.ts:612-631 (fake: 87) **Found:** 2026-10-05 by independent review of 8a.3's fixes (scope: 40f598e..dea9c59; lenses: quality, security, performance, tests) **Why it matters:** "a cancel while a same-person move's write waits for its retry leaves no event" fails the move's write with `postFails`, which answers 503 before the fake records the event, so the failed write leaves nothing in Ana's calendar and the cancel's removal has nothing to catch. On the code before the fix the move job never makes that call (it PATCHes, which succeeds), so no retry happens there at all; F-187's resolution notes the test passes on that code. The case it is named for is F-187's B2: the move job's first Google call fails after Google took it (a timeout, or a 5xx after the change), and the cancel lands before the retry. Probed in a scratch copy with that shape, failing the move job's first call whatever its method (taken, then answered 503): on the code before the fix the plain id stays in Ana's calendar for the cancelled booking; on this range none is left. The fix is still guarded (the "cancel during a same-person move's write" test fails if the cancel names no id), so this is the one saved case that adds nothing, not a gap in what is checked. **Suggested fix:** In that test, let the failed write be taken before it is refused (record the event, then answer 503), and fail the move job's first call whatever its method, so the same test fails on the code before the fix. **Resolution:** Fixed 2026-10-05: the fake Google can now take a write or update and then answer 503 (`takenThenFails`), and the test fails the move's own call that way, after the booking's held write. It passes on the current code and fails on 40f598e (the plain id stays in Ana's calendar for a cancelled booking). Closed 2026-10-05 by independent review of 8a.3's second fixes (scope: dea9c59..7f10bdc): rerun in a scratch copy of 40f598e with this range's test file, the test fails (Ana's calendar keeps one event for the cancelled booking); it passes on this range in 6 of 6 full runs. The fake now records the event before the 503 (move-booking-event.test.ts:92-94), and the counter's second call is the move job's own write, since the booking's held write is the first (the DELETE branch never asks `takenThenFails`). On this range it would fail if the cancel named any id but `s1`, the one the failed write left. The new quick-moves test fails on dea9c59 with 15 DELETE calls instead of 5, as reported.

8a/F-192 [P3] closed - Two lines of the move tests still describe the removal and the fake as they were before this range

File: backend/lib/calendar/move-booking-event.test.ts:274, 653 **Found:** 2026-10-05 by independent review of 8a.3's second fixes (scope: dea9c59..7f10bdc; lenses: quality, security, performance, tests)

Why it matters: F-190 asked for the comments stating the wider claim to be brought in line, and the three it named were; a fourth, in the tests, was missed: "an event never written gets written" still says "with no id saved, every id it could have is taken out", the sweep this range removed. It reads true only by accident (at move number 0 there is one id), and it tells a reader the removal still walks a list. In the retry test, the predicate `method !== "DELETE" && failures++ === 1` guards against a method that never reaches it: the fake asks `takenThenFails` only for a POST or a successful PATCH (lines 94 and 103), so the first half is always true and suggests DELETE calls are counted. Nothing fails because of either; both are text a reader trusts. **Suggested fix:** Say what happens now at line 274, for example "with no id saved, the id the booking's write would have used is taken out", and drop `method !== "DELETE" &&` at line 653 (or keep the fake's own comment as the only statement of which calls it covers). **Resolution:** Fixed 2026-10-05 as suggested: the comment now says the last write's id is taken out, and the retry predicate is `failures++ === 1`, since the fake asks it only for writes and updates. Closed 2026-10-05 by /audit independent current (scope: 779512a..dd65fe3): line 274 now reads "with no id saved, the last write's id is taken out", which is what move-booking.ts:280-290 does (one removal of `calendarEventIdOf(bookingId, row.sequence)`), and the test it sits in expects exactly one DELETE and one POST. Line 653 is `takenThenFails = () => failures++ === 1`; the fake calls `takenThenFails` only after a POST is taken (line 94) and after a successful PATCH (line 103), never for a DELETE, so nothing the old guard implied is lost. The repair (5d7f1bd) touches only those two lines, and the file passed in 6 of 6 full backend runs here.

Independent review

Status: passed **Target commit:** dd65fe3e1463bca51e130a587e8a29187e632913 **Base commit:** 779512aca1f2e67487f62fe33a34b928f2c3f97e **Base ref:** main **Spec hash:** 4ff18cc30070df14cd40d4d5a088d105a6d865753fc96b31e8a1059f55c4d878 **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-10-05T23:05:50Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-10-05T23:12:45Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Handoff

Review the active spec and the complete

779512aca1f2e67487f62fe33a34b928f2c3f97e..dd65fe3e1463bca51e130a587e8a29187e632913 delta

in a fresh session or isolated subagent without the builder conversation. Run all Audit lenses from scratch. Run Check when required above. Do not edit product code, accept findings, or reuse the existing findings as the review scope.

Commands

- `npm run build --workspace=backend` : pass
- `npm run test --workspace=@scheduleads-app/shared` : pass (17 files, 112 tests)
- `npm run test --workspace=backend` : pass, 6 of 6 full runs in a row (60 files, 599 tests each, 24 to 30 s)
- `npm run build --workspace=frontend` : pass (its prebuild also rebuilt the backend's route types)
- `npm run lint --workspace=frontend` : pass
- `npm run format:check` : pass

Evidence

- Freshness checked before reviewing: HEAD is dd65fe3e1463bca51e130a587e8a29187e632913, `git merge-base main HEAD` is 779512aca1f2e67487f62fe33a34b928f2c3f97e, current-feature.md hashes to 4ff18cc3...c4d878, and the only paths differing from the target were this file and the untracked owner's note blueprint/ai-voice-proposal.md (outside scope, not read).
- The whole delta was read (65 files, 15 commits): every file under backend/lib/jobs, the three booking changes, the three calendar functions, the four email senders, server.ts, the Vitest config and setup, the changed tests, package.json, and the plan and overview edits. The frontend is untouched by the range.
- Decisions 1 to 9 hold in code: jobs are added through `add_job` inside the caller's Drizzle transaction (enqueue-job.ts, book-time.ts:320-333, cancel-booking.ts:95-114, move-booking.ts:263-290); one job per email under its existing key; the jobs reread the booking and skip a moved confirmation, a replaced move, a cancelled write and anything after the start; 10 attempts with graphile-worker's `exp(attempts) waits` (about 3.6 hours in all); the calendar jobs run in `booking-event-<last two hex characters>`, 256 lanes for a UUID; a removal carries its person and event id; the runner starts before `serve` and stops on SIGTERM and SIGINT.
- Calendar races traced by hand against the lane lock: a write or move whose booking changes during its Google call saves its id only while person, move number and status still hold (write-booking-event.ts:87-100), and every change that finds no id saved names the one id that write could have used; a write retried after a cancel or later move is skipped (is-booking-event-job-due.ts, write-booking-event.ts:64). No path found that leaves an event behind or writes an id this booking deleted.
- Security: payloads are ids and a sequence only; a task's error is reduced by `safeErrorReason` before graphile-worker stores or logs it (job-task.ts); the logger prints warnings and errors, first line only, plus

the booking id; schema names go through `sql.identifier`. No route or answer changed. The tests' fetch is kept to localhost by `vitest.setup.ts`.

- `graphile-worker 0.18.0` read where the code depends on it: `job:failed` fires only when attempts reach `max_attempts` (`dist/worker.js:239-252`); idle workers poll every `pollInterval` (`dist/worker.js:146-158`); `run` and `runOnce` migrate their schema themselves (`dist/lib.js:323-326`).
- After the six full backend runs, a read-only count of `scheduleads_dev` showed 0 jobs, 0 locked jobs, 0 locked lanes, 0 `test-%` tasks and 0 `test-%` queues in `graphile_worker` and in all 15 `graphile_worker_test_N` schemas. 20 `test-%` organizations remain, every one created on 2026-10-03 or 2026-10-04, before this review's runs.
- F-192 re-examined and closed (`move-booking-event.test.ts:274` and `653`).

Findings

- F-193 [P3] open: a deploy's stop exits without waiting for requests in flight (`backend/server.ts:39-48`)
- F-194 [P3] open: `installJobTables` has no production caller and its comment misstates how the tests install the tables (`backend/lib/jobs/install-job-tables.ts`)
- F-192 [P3] closed by this pass
- No P0 or P1 is open or fixed in the ledger

Remaining risk

- F-176 [P2] stays open: `RAILWAY_DEPLOYMENT_DRAINING_SECONDS` must be set on the Railway backend before the deploy that ships 8a, or a job in flight at a deploy waits about four hours for its lock. Frank's change, at that deploy.
- The `SIGTERM` path in `server.ts` has never run (Windows cannot send the signal); it is first seen on Railway.
- The API started by hand working a real job was not rechecked here: dev servers were out of bounds for this review. The jobs are proved through the tests' helper and the runner's own tests only.
- An idle API's five workers each query for a job every 2 seconds (`graphile-worker`'s default), about 2.5 queries a second against Railway's Postgres with no traffic. Unmeasured; the library's local queue option would reduce it if the cost shows.
- No dependency vulnerability scan was run for `graphile-worker` and its new packages (`pg`, `graphile-config` and others): no network access in this review.
- F-179 and F-153 stay unverified, carried from earlier reviews.
- Blueprint activity state (`blueprint/.state/run.json`) was not written by this reviewer, which was limited to the two review files.